Assistant RAG sur les démarches administratives du Sénégal

Rapport de présentation

Formation	Certification IA — Orange Digital Center (P1IA)
Groupe	Groupe 1
Membres	Alassane (lead / intégration), Tayba (corpus), Fatima (retrieval), Khadidiatou (évaluation), Abdou Aziz (fonctionnalités avancées)
Date	25 octobre 2026

1. Objectif

Construire un assistant capable de répondre à des questions sur un corpus de documents **publics en français**, en **citant les passages utilisés**, puis de **mesurer la qualité** de ses réponses. Corpus retenu : les **démarches administratives du Sénégal** (carte nationale d'identité, passeport, état civil, casier judiciaire, certificat de résidence, permis de conduire).

2. Corpus

— **8 documents** texte regroupés par démarche dans `data/corpus/`, découpés en **37 segments** (*chunks*) indexés.
— **Sources publiques**, citées et tracées dans `data/sources.json` : guide des démarches et dossier passeport (Senego), fiche CNI (OSIRIS / Le Soleil), article e-Senegal (NDARINFO).
— Aucune donnée personnelle ni confidentielle.
— Point notable : sur le casier judiciaire, le corpus documente *deux procédures concurrentes* (dépôt physique au greffe *vs* plateforme e-Senegal en déploiement), utile pour tester la nuance.

3. Architecture du pipeline

Le système se compose de deux phases : une **indexation** hors-ligne (le corpus est transformé en base vectorielle) et une **interrogation** en ligne (la question est confrontée à l'index puis au modèle de langage).

4. Flux intégré (fonctions avancées)

Une seule fonction, `answer_advanced()`, enchaîne les garde-fous, l'appel d'outils éventuel, le cœur RAG et l'observabilité.

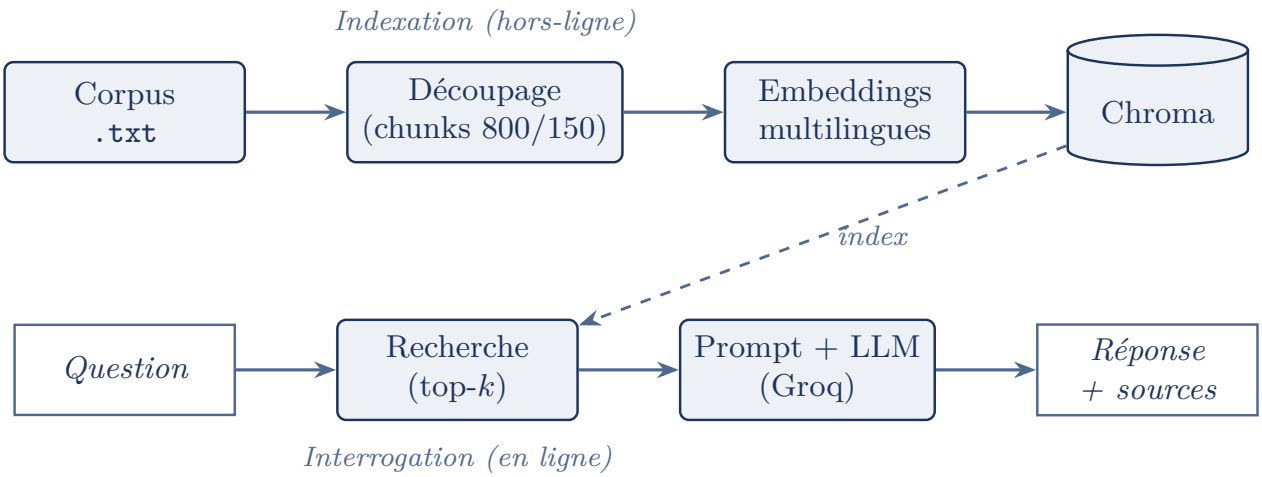


FIGURE 1 – Pipeline RAG : indexation du corpus (haut) et interrogation avec citation des sources (bas).

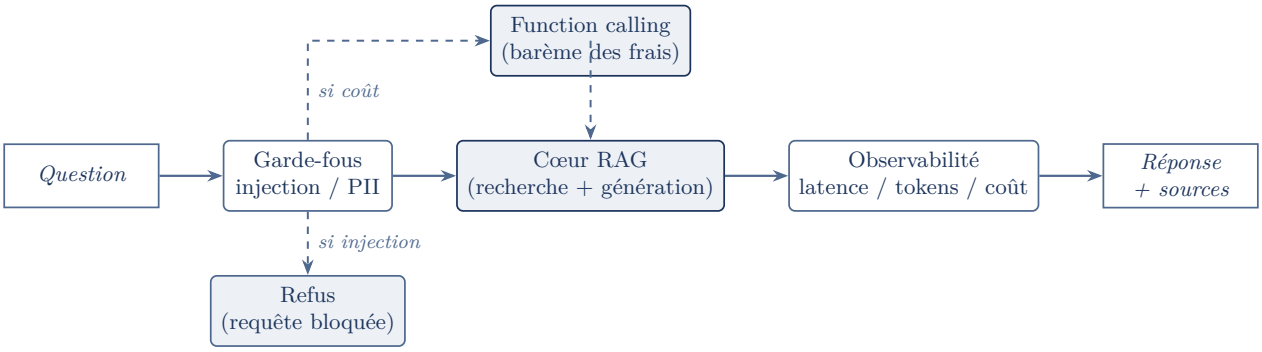


FIGURE 2 – Flux `answer_advanced()` : sécurité en amont, outils métier, RAG, puis mesure.

5. Choix techniques (justifiés)

Composant	Choix	Justification
Framework	LangChain	standard, composable (loader, splitter, retriever, LLM).
Embeddings	MiniLM multilingue	gratuit, bon en français, léger.
Base vectorielle	Chroma (persistée)	simple, local, gratuit, relançable.
Découpage	800 car. / recouvrement 150	compromis contexte / précision du passage.
LLM	Groq `gpt-oss-20b`	gratuit, rapide, bon en français.
Garde-fous	règles + seuil + masquage	sécurité sans coût LLM additionnel.

Aucun budget requis : API Groq gratuite et modèle d'embeddings ouvert exécuté localement.

6. Protocole d'évaluation

Jeu de **15 questions** (`eval/questions.json`) : **12 factuelles** (avec réponse de référence et source attendue) et **3 pièges** (hors-sujet, injection de prompt, données personnelles). Exécution : `python -m src.evaluation.run_eval`. Trois mesures :

— **Pertinence du retrieval** : la source attendue figure-t-elle dans les passages renvoyés ? (objectif, automatique)

— **Exactitude** : la réponse contient-elle les informations clés attendues ?
— **Garde-fous** : hors-sujet refusé, injection bloquée, données personnelles masquées.

7. Résultats

Métrique	Score
Pertinence du retrieval	11 / 12 (91%)
Exactitude (mots-clés)	10 / 12 (83%)
Garde-fous (pièges)	3 / 3 (100%)
Latence typique	≈ 0,6 à 3 s / requête
Coût estimé	≈ 0,0001 à 0,0003 $ / requête

Les trois pièges sont correctement gérés : hors-sujet → « Je ne sais pas d'après le corpus », injection → requête bloquée, numéro d'identité → masqué avant traitement.

8. Analyse des échecs

Deux « échecs » seulement, et l'analyse est instructive.

(1) `passeport_docs` — véritable échec de retrieval. La question « Quels documents pour une première demande de passeport adulte ? » reçoit « Je ne sais pas d'après le corpus ». Le dossier passeport étant long, il génère beaucoup de segments ; pour cette formulation, les passages « documents requis » d'autres démarches sont sémantiquement proches et le bon segment passeport ne remonte pas dans le top-k. C'est une *confusion inter-documents* classique en RAG. *Pistes* : augmenter k, filtrer par métadonnée « démarche », ou affiner le découpage.

(2) `passeport_prix` — faux échec (artefact de mesure). La réponse est en réalité correcte (« 20000 FCFA », via l'outil fiscal) mais comptée comme fausse car l'outil écrit « 20000 » sans séparateur, alors que le mot-clé attendu est « 20 000 ». Notre métrique par mots-clés est donc *sensible au formatage* : après revue manuelle, **11 réponses sur 12 sont réellement correctes**.

Enseignement. La qualité dépend autant du *retrieval* (le bon passage doit remonter) que de la *mesure* (une métrique trop littérale sous-estime la performance) : il faut croiser métrique automatique et relecture.

9. Limites et améliorations

Limites : corpus restreint (8 documents, couverture partielle) ; pas de filtrage par démarche au retrieval ; métrique d'exactitude approximative ; dépendance à un modèle gratuit.

Améliorations : filtre par métadonnée « démarche » ou *reranking* ; test d'autres embeddings et tailles de segment (via `src/rag/config.py`) ; enrichissement du corpus ; évaluation complémentaire avec RAGAS (faithfulness, context precision/recall).

10. Fonctionnalités avancées (implémentées)

— **Function calling** : outil « barème officiel » (frais, timbres, pièces par démarche), déclenché sur les questions de coût.
— **Observabilité** : latence, tokens, coût estimé par requête, journalisés dans `eval/traces.jsonl`.
— **Garde-fous** : blocage des injections de prompt, masquage des données personnelles (NIN, téléphone, courriel), refus hors-sujet.

11. Répartition du travail

Membre	Contribution
Alassane	Cœur RAG LangChain, intégration, pipeline unifié.
Tayba	Corpus (collecte, nettoyage) et provenance des sources.
Fatima	Réglage du retrieval (modèle d'embeddings, segments, k).
Khadidiatou	Jeu d'évaluation, métriques, analyse des échecs.
Abdou Aziz	Garde-fous, observabilité, function calling.

12. Plan de démonstration (≈ 5 min)

1. Problème et corpus (30 s).
2. Questions en direct : une factuelle (réponse + sources), une de coût (outil), un piège d'injection (blocage) (2 min).
3. Résultats d'évaluation (1 min).
4. L'échec `passeport_docs` et les limites (1 min).
5. Observabilité (latence / coût) en direct (30 s).

Évaluation réalisée le 25 octobre 2026 sur 15 questions — reproductible via `python -m src.evaluation.run_eval` (voir `eval/results.md`).